Progressive Student Dashboard — Final Project Submission & Technical Document

Repository URL: https://github.com/Gunajee/progressive-student-dashboard
Author / Developer: Full-Stack Engineering Candidate
Target Platform: Web (Desktop & Responsive Mobile)

---

1. Executive Summary

Problem Statement

In self-paced online education, students often experience high drop-out rates and loss of motivation due to a lack of feedback loops, progress tracking, and structured guidance. Simultaneously, mentors and course facilitators lack real-time visibility into student engagement, making early intervention for struggling or inactive students difficult.

Proposed Solution

The **Progressive Student Dashboard** is an enterprise-grade full-stack web application designed to solve these pain points. It combines:

- **Gamified Student Analytics**: Visual progress indicators, daily study streaks, time-series learning trends, and completion distribution charts.
- **Explainable Recommendation Engine**: Priority-based next steps guiding students to resume courses, review completed topics, or seek mentor support.
- **Proactive Mentor Monitoring**: Real-time cohort health metrics, search/filter student directories, at-risk student classification, and single-click RFC-compliant CSV report exports.
- **Admin Course Management**: Full CRUD portal allowing course creation, lesson reordering, and authoring rich-text lesson content using GitHub-Flavored Markdown.

---

2. System Architecture & Tech Stack

```text
React 18 + Vite (Port 5173)
%
%¼ (REST API over JWT Bearer Token)
Spring Boot 3.3.3 API Server (Port 8080)
%
% % % % % % % % % % % % % %4% % % % % % % % % % % % %
%¼ %¼
Spring Security 6 Service Logic Layer
(BCrypt & JWT Filter) %
%¼
Spring Data JPA / Hibernate
%
%¼
MySQL Database / H2 Database
```

Technology Stack Table

Layer	Technologies Used	Key Responsibilities
Frontend	React 18, Vite, Tailwind CSS (v4), Recharts, Lucide Icons, React Markdown	Single Page App

(SPA), responsive UI, interactive trend/pie charts, markdown rendering |
Backend	Java 21, Spring Boot 3.3.3, Spring Security 6, Spring Data JPA, Maven	REST API endpoints, JWT authentication, business logic, analytics calculation, CSV generation
Database	MySQL 8.0 / In-memory H2	Relational data storage, indexing on search & foreign keys
Testing	JUnit 5, Mockito, MockMvc, Vitest, React Testing Library	Automated backend unit/integration tests and frontend component tests

---

3. Core Roles & Feature Breakdown

Ø=Üh Ø<ß“ 3.1 Student Console

1. **Interactive Dashboard**:
   • **Summary Cards**: Enrolled courses count, lessons finished count, total study hours logged, overall progress percentage, and an animated daily learning streak badge.
   • **Learning Trend Chart**: Time-series Area Chart (powered by Recharts) with 7-day and 30-day range filters showing daily study minutes.
   • **Course Distribution Chart**: Donut Chart showing completion status distribution (Completed, In Progress, Not Started).
   • **Recent Activity Feed**: Chronological history of completed lessons and study milestones.
2. **Adaptive Recommendation Cards**:
   • Priority-ranked suggestions (`HIGH`, `MEDIUM`, `LOW`) with explainable context (e.g. *"You completed Lesson 1, resume Lesson 2 in React Fundamentals"*).
3. **Course Syllabus & Lesson View**:
   • Expandable lesson view displaying estimated study time, logged study minutes, and rich-text Markdown lesson material.
   • Interactive study time logging tool and progress status toggles (`NOT_STARTED` '" `IN_PROGRESS` '" `COMPLETED`).

Ø=Üh Ø<ßë 3.2 Mentor Directory & At-Risk Detection

1. **Cohort Health Summary**: High-level metrics tracking total students, active students, cohort average progress, and total at-risk students.
2. **Student Directory & Filtering**:
   • Search students by Name or Email keywords.
   • Filter cohort by status (`HEALTHY`, `NEEDS_ATTENTION`, `AT_RISK`).
   • Server-side pagination.
3. **Individual Student Monitoring**:
   • View detailed analytics for any student, including their specific learning trend, course progress, last active timestamp, and risk status.
4. **RFC-4180 CSV Export**:
   • Download complete student metrics report as a formatted `.csv` file.

Ø=Þàþ 3.3 Admin Course Management & Content Authoring

1. **Course CRUD Operations**: Create, edit, and delete courses with customizable titles, categories, difficulty levels (`BEGINNER`, `INTERMEDIATE`, `ADVANCED`), and estimated durations.
2. **Lesson Syllabus Builder**: Create and edit lessons for any course, set order indexes, and specify study estimates.
3. **Markdown Split-Pane Editor**: Integrated live-preview Markdown editor allowing administrators to format lesson material with headings, lists, code blocks, and blockquotes.

---

4. Database Schema (Entity Relationship)

```
```text
% % % % % % % % % % % % % % % % % % % %          % % % % % % % % % % % % % % % % % % % %
%      users      %  1         * %    enrollments    %
% % % % % % % % % % % % % % % % % % % %$% % % % % % % % % % % % % % % %<% % % % % % % % % %
%  id (PK)        %             %  id (PK)         %
```
```

```
%  email (UNIQUE)   %              %  student_id (FK)  %
%  password_hash    %              %  course_id (FK)   %
%  name             %              %  enrolled_at      %
%  role             %              % % % % % % % % % % % % % % % % % % % % % % % % % % %
%  mentor_id (FK)   %
% % % % % % % % % %,% % % % % % % % % % %
%  1
%
%¼ *
% % % % % % % % % % % % % % % % % % % %              % % % % % % % % % % % % % % % % % % % % % % %
%  lesson_progress  %  *          1 %      lessons      %
% % % % % % % % % % % % % % % % % % % %$% % % % % % % % % % % % % % % %<% % % % % % % % % % %
%  id (PK)          %              %  id (PK)          %
%  student_id (FK)  %              %  course_id (FK)   %
%  lesson_id (FK)   %              %  title            %
%  status           %              %  description      %
%  time_spent_mins  %              %  content (TEXT)   %
%  last_updated     %              %  order_index      %
% % % % % % % % % % % % % % % % % % % % %              %  est_minutes      %
% % % % % % % % % %,% % % % % % % % % % %
%  *
%
%¼ 1
% % % % % % % % % % % % % % % % % % % % %
%      courses      %
% % % % % % % % % % % % % % % % % % % % %$
%  id (PK)          %
%  title            %
%  category         %
%  difficulty       %
%  est_hours        %
% % % % % % % % % % % % % % % % % % % % %
```

---

5. Security & Data Protection Audit

Security Domain	Implementation Standard
Password Hashing	BCrypt password encoder with standard strength factor. Passwords are never stored or returned in plain text.
Authentication	Stateless JWT tokens signed with SHA-256 secret keys. Tokens sent via `Authorization: Bearer <token>` HTTP headers.
Authorization	Role-based URL protection using Spring Security rules: `/api/student/**` (STUDENT), `/api/mentor/**` (MENTOR), `/api/admin/**` (ADMIN).
Student Isolation	Students can only query and mutate their own progress data; unauthorized access to other students' endpoints yields `403 Forbidden`.
Input Protection	Parameterized JPA queries prevent SQL Injection. DTO validation prevents malformed payloads.
CORS Policy	Restricted CORS policy configured for `http://localhost:5173` with credentials support.

---

6. Seed Credentials for Testing

Role	Email	Password	Access Rights
Admin	`admin@example.com`	`password`	Course & Lesson Management, Markdown Editor
Mentor	`mentor@example.com`	`password`	Cohort Directory, Student Risk Monitoring, CSV Export
Student	*(Register via `/register`)*	*(Any password)*	Personal Dashboard, Analytics, Course Syllabus, Lesson Completion

---

7. How to Run Locally

1. Backend Launch (Spring Boot)

```bash
cd backend

# Runs on Port 8080 with in-memory H2 database & automatic seed data

./mvnw spring-boot:run
```

2. Frontend Launch (React + Vite)

```bash
cd frontend
npm install
npm run dev

# Open http://localhost:5173 in browser

```

3. Running Automated Tests

```bash
# Backend Tests (JUnit 5 + Mockito)

cd backend && ./mvnw clean test

# Frontend Tests (Vitest + React Testing Library)

cd frontend && npm test
```

---

8. Conclusion & Deliverables

The **Progressive Student Dashboard** fully delivers on all requirements:

1. Full-stack modular architecture (Spring Boot 3 + React 18).
2. Complete Auth, Analytics, Recommendations, Mentor Cohort Export, and Admin Course Management capabilities.
3. High visual standards with responsive charts and dark/light styled components.
4. Comprehensive test coverage and clear documentation.

GitHub Repository: https://github.com/Gunajee/progressive-student-dashboard